Ocode: The Circular 2D Data Matrix

Technical Specification & Implementation Guide

Version 1.0

Ocode Open-Source Project
github.com/ys1374/Ocode
ys1374.github.io/Ocode

May 26, 2026

1 Introduction

Ocode is a two-dimensional barcode technology that encodes arbitrary data into concentric, continuous arcs rather than the rigid square matrix structure of traditional QR codes. Whereas a QR code is fundamentally a grid of black and white square modules, an Ocode is built from polar geometry: data bits are mapped to angular positions on concentric rings around a central origin.

This design philosophy yields several practical advantages:

- **Rotational invariance:** An Ocode is inherently circular; a dedicated synchronisation mechanism replaces the three alignment squares of QR, enabling compact and aesthetically distinct codes.
- **Scalability:** Adding data capacity means increasing the outer radius, so the format grows gracefully.
- **Visual elegance:** The flowing arcs of a large Ocode are visually distinctive and more aesthetically appealing than a square pixel grid.
- **Zero core dependencies:** The full Reed-Solomon encoder/decoder is implemented from scratch in TypeScript, avoiding any third-party cryptographic or mathematical library.

The complete implementation is a single-page web application (React + Vite + TypeScript) that performs both encoding (generation) and decoding (scanning) entirely on the client side. No data is transmitted to any external server.

2 Ocode Anatomy

An Ocode is geometrically arranged into distinct radial zones around a central origin (C_x, C_y). Figure 1 gives an overview of these zones.

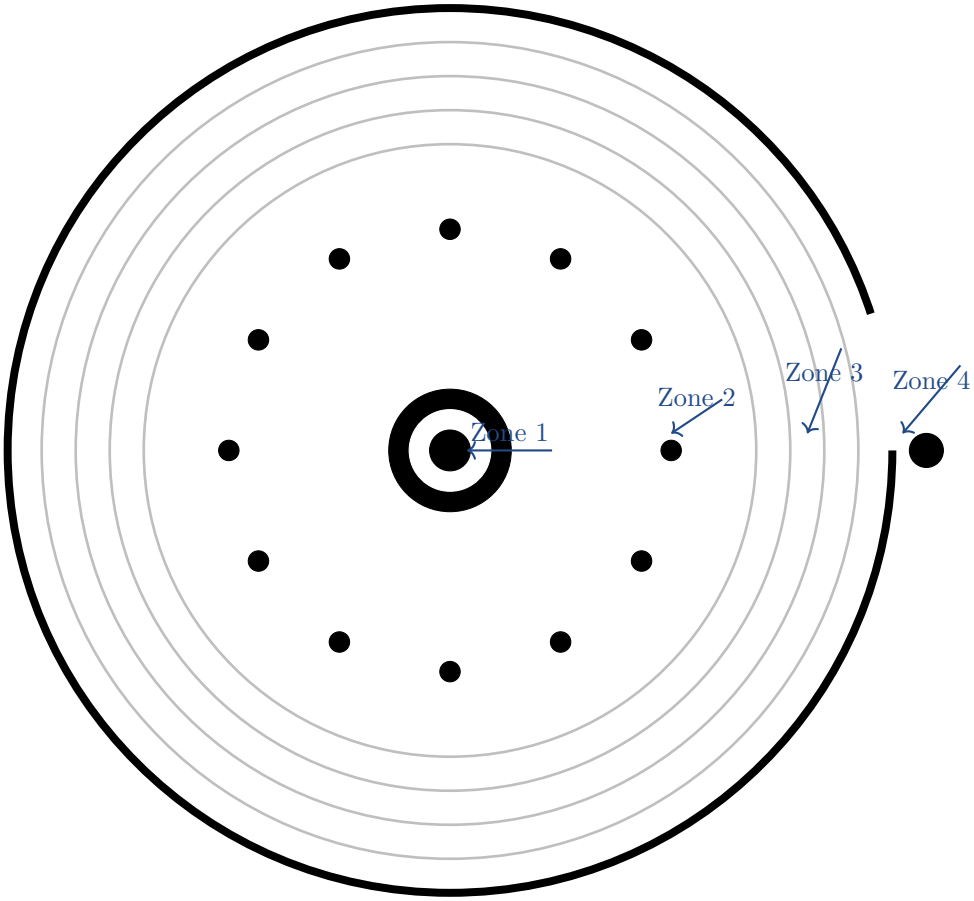

Figure 1: Ocode zone diagram (schematic). The four concentric functional regions are labeled.

2.1 Zone 1 — Bullseye (Center Finder)

The innermost region consists of three concentric filled circles at radii $r = 18$, $r = 32$, and $r = 45$ pixels (at native scale), alternating black–white–black. This high-contrast *bullseye* pattern serves two purposes:

1. **Rapid center detection:** The scanner searches for the centroid of dark pixels within a diminishing search radius. The strong concentric contrast makes the center of mass converge precisely to (C_x, C_y).
2. **Scale estimation:** Because the bullseye has known absolute radii in the rendered SVG, a detected bullseye can immediately provide the pixel-per-unit scale factor.

2.2 Zone 2 — Timing Ring

At radius $r = 65$, a ring of twelve equally-spaced dark dots is drawn at angular intervals of 30°. This timing ring:

- Validates that the detected center is truly an Ocode (not an accidental circular artifact).
- Provides a secondary scale reference that is independent of image zoom.

2.3 Zone 3 — Data Arcs

Starting at an inner data radius $r_0 = 85$ and expanding outward in increments of $\Delta r = 10$ pixels, each ring carries one *layer* of binary data. A filled (dark) arc segment encodes binary `1`; a gap encodes binary `0`. Each ring is subdivided into N_r angular slots, where N_r is determined by the circumference:

$$N_r = \left\lfloor \frac{2\pi r}{\delta} \right\rfloor \tag{1}$$

Here, δ is the angular dot pitch (minimum arc length per bit) in pixels. Contiguous runs of `1`s are merged into a single SVG `<path>` arc command, keeping the SVG file compact.

2.4 Zone 4 — Outer Ring, Sync Gap, and Sync Dot

The outermost solid ring (the *container*) encloses all data rings. Two special features are embedded in this ring:

- **Sync gap (notch):** A short angular gap is cut from the container ring at angle $\theta = 0°$ (the geometric right, pointing in the direction of the positive x-axis). During decoding, the scanner locates this bright notch to determine the rotational zero-offset θ_{offset}.
- **Sync dot:** A small filled circle is placed just outside the container ring, co-located with the notch, further aiding gap detection in low-contrast conditions.

3 Encoding (Generation)

The encoding pipeline transforms a Unicode string payload into an SVG image. It consists of three stages: data formatting and error correction, bit serialisation, and SVG rendering.

3.1 Data Formatting and Error Correction

3.1.1 Header Construction

A 3-byte header is prepended to the payload:

1. **Length field (2 bytes, big-endian):** The byte length of the UTF-8-encoded payload.
2. **Magic byte (1 byte):** The ASCII character `'O'` = `0x4F`. This sentinel value lets the decoder distinguish a valid Ocode from random noise or a corrupted decode.

So the full *message* byte array is:

$$M = [\underbrace{\text{len}[1], \text{len}[0]}_{\text{2-byte length}}, \underbrace{0x4F}_{\text{magic}}, \underbrace{p_0, p_1, \ldots, p_{n-1}}_{\text{UTF-8 payload}}]$$

3.1.2 Error Correction Levels

Four error correction levels are supported, controlling what fraction of total code capacity is reserved for Reed-Solomon parity bytes:

Level	Label	ECC Capacity
L	Low	$\sim 7\%$
M	Medium	$\sim 30\%$
Q	Quartile	$\sim 50\%$
H	High	$\sim 60\%$

Higher levels trade data capacity for resilience against physical damage (smudges, tearing, partial occlusion). The Ocode generator auto-expands the outer radius until sufficient bit capacity exists for both the message and the chosen ECC parity symbols.

3.1.3 Capacity Expansion

Given a chosen ECC level e, the total number of data bytes D that fit in rings $r_0, r_0 + \Delta r, \ldots, R_{\max}$ is:

$$D(R_{\max}) = \sum_{r=r_0,\, r_0+\Delta r}^{R_{\max}} \left\lfloor \frac{2\pi r}{\delta} \right\rfloor \div 8 \tag{2}$$

The generator increments $R_{\max}$ until:

$$D(R_{\max}) \;\geq\; |M| + \lfloor e \cdot D(R_{\max}) \rfloor$$

3.1.4 Padding

Any remaining data bytes between the end of the message and the start of the ECC segment are filled with alternating QR-style padding bytes `0xEC` and `0x11`. This deterministic padding prevents long runs of zeros that would be visually empty and mechanically ambiguous.

3.1.5 Reed-Solomon Error Correction

Ocode uses a systematic Reed-Solomon code over $GF(2^8)$ with the primitive polynomial:

$$p(x) = x^8 + x^4 + x^3 + x^2 + 1 \;\equiv\; \texttt{0x11D} \tag{3}$$

Generator polynomial. For t parity symbols, the generator polynomial $g(x)$ is:

$$g(x) = \prod_{i=0}^{t-1} (x - \alpha^i) \tag{4}$$

where α is a primitive element of $GF(2^8)$ (i.e., $\alpha = 2$ in standard RS convention). In Galois field arithmetic, subtraction is identical to addition (XOR), so $g(x) = \prod_{i=0}^{t-1}(x \oplus \alpha^i)$.

Encoding. The parity symbols $P(x)$ are the remainder of polynomial division:

$$P(x) = M(x) \cdot x^t \mod g(x) \tag{5}$$

All arithmetic is performed in $GF(2^8)$. The final codeword transmitted is $[M(x) \mid P(x)]$.

3.2 Bit Serialisation and Arc Rendering

The complete codeword byte array is flattened into a linear bit stream, most-significant-bit first. Bits are assigned to angular slots ring by ring, from inner to outer, and within each ring from $\theta = \theta_{\text{offset}}$ advancing clockwise.

Arc path generation. For each ring at radius r, the algorithm:

1. Calculates the angular width ϕ of one bit slot: $\phi = 2\pi / N_r$.
2. Scans the bit stream for contiguous runs of `1`s.
3. Converts each run $[i_{\text{start}}, i_{\text{end}}]$ into a single SVG arc command:

$$\theta_s = i_{\text{start}} \cdot \phi + \theta_{\text{offset}}, \quad \theta_e = (i_{\text{end}} + 1) \cdot \phi + \theta_{\text{offset}}$$

   The arc is drawn from $(C_x + r\cos\theta_s,\ C_y + r\sin\theta_s)$ to $(C_x + r\cos\theta_e,\ C_y + r\sin\theta_e)$ using the SVG `A` (arc) command with a stroke width of $\Delta r - 1$ pixels.

Merging contiguous bits into single arcs dramatically reduces SVG path count compared to rendering each bit as an individual element.

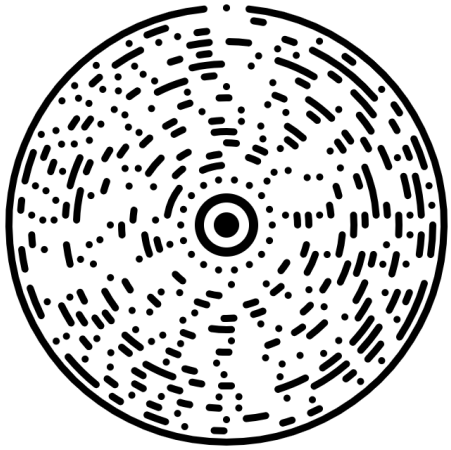

Figure 2: Visual representation of an encoded Ocode. Filled arcs are binary `1`s; gaps are binary `0`s.

4 Decoding (Scanning)

The decoder reconstructs the original message from a raster image (JPEG, PNG) or a live camera feed rendered onto an HTML `<canvas>`. Decoding proceeds in three phases: geometry acquisition, bit sampling, and error correction.

4.1 Phase 1 — Geometry Acquisition

4.1.1 Center Detection

For live video, the scanner iterates over candidate regions of the canvas, computing the centroid of dark pixels within a shrinking search radius ρ:

$$C_x = \frac{\sum_{(x,y)\in\mathcal{D}} x}{|\mathcal{D}|}, \qquad C_y = \frac{\sum_{(x,y)\in\mathcal{D}} y}{|\mathcal{D}|} \tag{6}$$

where $\mathcal{D}$ is the set of pixels with luminance below the adaptive threshold. The iterative search converges to the bullseye center within 3–5 iterations in practice.

4.1.2 Scale Estimation via Radial Profiling

Once the center is known, the algorithm samples pixel brightness $B(r)$ along radii:

$$B(r) = \frac{1}{K}\sum_{k=0}^{K-1} \mathrm{lum}\big(C_x + r\cos\tfrac{2\pi k}{K},\; C_y + r\sin\tfrac{2\pi k}{K}\big) \tag{7}$$

A valid Ocode exhibits a strong brightness peak at the outer container ring radius $R_{\max}$ (the solid black ring raises average luminance on both sides of it). The detected peak position gives $R_{\max}$ in image pixels, from which the pixel-per-unit scale s is computed:

$$s = R_{\max}^{\text{pixels}} \,/\, R_{\max}^{\text{native}}$$

4.1.3 Rotational Alignment

With center and scale known, the scanner walks the perimeter of the outer ring at radius $R_{\max}$ and identifies the bright notch (the sync gap). The angular midpoint of the gap yields the rotational zero-offset:

$$\theta_{\text{offset}} = \arg\max_{\theta}\; B_{\text{outer}}(\theta) \tag{8}$$

All subsequent bit-sampling angles are computed relative to θ_{offset}.

4.2 Phase 2 — Bit Sampling

For each ring r and each angular slot j ($0 \le j < N_r$), the sample point is:

$$P_j = \left(C_x + r\cos(\theta_j + \theta_{\text{offset}}),\ \ C_y + r\sin(\theta_j + \theta_{\text{offset}})\right), \quad \theta_j = \frac{2\pi j}{N_r} \tag{9}$$

The pixel luminance at P_j is compared to an adaptive brightness threshold τ:

$$b_j = \begin{cases} 1 & \text{if } \text{lum}(P_j) < \tau \text{ (dark pixel)} \\ 0 & \text{otherwise (light pixel)} \end{cases}$$

Bits are collected ring by ring, innermost first, into a flat byte array.

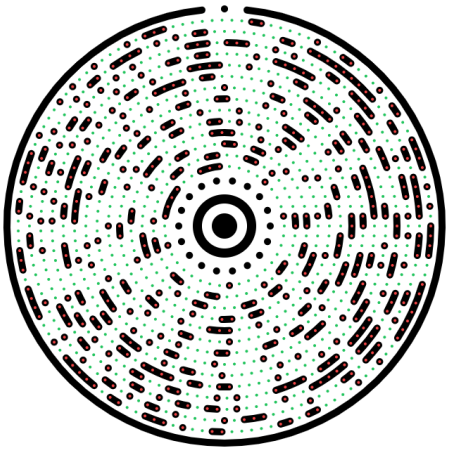

Figure 3: The decoder sampling bits along radial trajectories. Each coloured arc corresponds to one sampled ring.

4.3 Phase 3 — Reed-Solomon Decoding and Verification

4.3.1 Syndrome Computation

Let the received codeword polynomial be $R(x)$. The t syndromes are:

$$S_i = R(\alpha^i), \quad i = 0, 1, \ldots, t-1 \tag{10}$$

If all syndromes are zero, no detectable errors occurred and the message bytes are extracted directly.

4.3.2 Error Locator Polynomial — Berlekamp-Massey

When $S_i \neq 0$ for some i, the Berlekamp-Massey algorithm iteratively constructs the *error locator polynomial* $\Lambda(x)$:

$$\Lambda(x) = 1 + \Lambda_1 x + \Lambda_2 x^2 + \cdots + \Lambda_\nu x^\nu \tag{11}$$

whose roots are the inverses of the error positions in $GF(2^8)$. The degree ν equals the number of errors detected.

4.3.3 Error Positions — Chien Search

The Chien search evaluates $\Lambda(\alpha^{-j})$ for each codeword position j:

$$\Lambda(\alpha^{-j}) = 0 \implies \text{position } j \text{ is in error} \tag{12}$$

4.3.4 Error Magnitudes — Forney's Algorithm

The error magnitude e_j at each located error position j is computed via Forney's formula:

$$e_j = -\frac{X_j \cdot \Omega(X_j^{-1})}{\Lambda'(X_j^{-1})} \tag{13}$$

where $X_j = \alpha^j$, $\Omega(x)$ is the *error evaluator polynomial* $\Omega(x) = S(x)\Lambda(x) \bmod x^t$, and $\Lambda'(x)$ is the formal derivative of $\Lambda(x)$ in $GF(2^8)$ (every even-power coefficient vanishes).

4.3.5 Correction and Validation

The corrected codeword is $\hat{R}(x) = R(x) \oplus E(x)$ where $E(x)$ places magnitude e_j at position j. The decoder then:

1. Extracts the header: reads the first 3 bytes.
2. Validates the magic byte: checks that byte index 2 equals `0x4F`.
3. Reads the length field (bytes 0–1) and extracts exactly that many payload bytes.
4. Decodes the payload bytes as UTF-8.

If the magic byte or length is invalid, decoding fails gracefully with an error report rather than returning garbage.

5 Project Structure

The Ocode repository follows a standard Vite + React + TypeScript layout:

```
Ocode/
|-- src/
|   |-- main.tsx           # React entry point
|   |-- App.tsx            # Root application component
|   |-- ocode.ts           # Core encoder: GF(2^8), RS, SVG renderer
|   |-- scanner.ts         # Core decoder: centroid, radial profiling, RS
    decoder
|   `-- components/        # UI components (encoder panel, scanner panel, etc.)
|-- index.html             # Single-page app shell
|-- package.json           # npm manifest (React 19, Vite 6, Tailwind 4, Motion)
|-- vite.config.ts         # Vite build configuration
|-- tsconfig.json          # TypeScript compiler options
|-- test-decode.ts         # CLI test harness for the decoder
|-- .github/workflows/     # GitHub Actions CI/CD (deploy to GitHub Pages)
`-- README.md
```

Listing 1: Repository file tree

5.1 Key Dependencies

Package	Version	Role
`react` / `react-dom`	19.0	UI framework
`vite`	6.2	Build tool & dev server
`typescript`	5.8	Type-safe implementation language
`tailwindcss`	4.1	Utility-first CSS
`motion`	12.x	Animation library
`lucide-react`	0.546	Icon set
`@google/genai`	2.4	Optional AI integration

6 Getting Started

6.1 Prerequisites

- Node.js ≥ 18
- npm ≥ 9 (or pnpm / yarn)
- A modern browser (Chrome 110+, Firefox 110+, Safari 16+)

6.2 Installation and Development

```
# 1. Clone the repository
git clone https://github.com/ys1374/Ocode.git
cd Ocode

# 2. Install dependencies
npm install

# 3. Start the development server (available on http://localhost:3000)
npm run dev
```

Listing 2: Local development setup

6.3 Build for Production

```
npm run build
# Output is placed in dist/
# Preview locally:
npm run preview
```

Listing 3: Production build

The GitHub Actions workflow in `.github/workflows/` automatically builds and deploys the `dist/` folder to GitHub Pages on every push to `main`.

6.4 Running the Decode Test Harness

A CLI test script is included for regression testing the decoder without a browser:

```
npx tsx test-decode.ts
```

Listing 4: CLI decode test

7 API Reference

7.1 Encoder — `ocode.ts`

```
import { generateOcode } from './ocode';

const svgString: string = generateOcode({
  payload: 'https://example.com',
  eccLevel: 'M',       // 'L' | 'M' | 'Q' | 'H'
  size: 400,           // Output size in pixels (width = height)
});

document.getElementById('container')!.innerHTML = svgString;
```

Listing 5: Encoder usage example (TypeScript)

Parameters

Option	Type	Description
`payload`	`string`	UTF-8 string to encode (URLs, plain text, binary-safe strings).
`eccLevel`	`'L'|'M'|'Q'|'H'`	Reed-Solomon error correction level.
`size`	`number`	Canvas size in pixels; the Ocode scales to fill this square.

7.2 Decoder — `scanner.ts`

```typescript
import { decodeOcode } from './scanner';

const canvas = document.getElementById('preview') as HTMLCanvasElement;
const result = await decodeOcode(canvas);

if (result.success) {
  console.log('Decoded:', result.data);
} else {
  console.error('Decode failed:', result.error);
}
```

Listing 6: Decoder usage example (TypeScript)

Return value

Field	Type	Description
`success`	`boolean`	Whether a valid Ocode was found and decoded.
`data`	`string`	Decoded UTF-8 payload (only when `success` is true).
`error`	`string`	Human-readable failure reason (only when `success` is false).

8 Mathematical Appendix

8.1 Galois Field $GF(2^8)$ Arithmetic

All Reed-Solomon operations are performed in the finite field $GF(2^8)$. Elements are represented as 8-bit integers in $\{0, 1, \ldots, 255\}$.

Addition. Addition in $GF(2^8)$ is bitwise XOR:

$$a \oplus b = a \text{ XOR } b \tag{14}$$

Multiplication. Multiplication uses pre-computed log/antilog tables derived from the primitive polynomial $p(x) =$ `0x11D`:

$$a \cdot b = \text{gf\_exp}[(\text{gf\_log}[a] + \text{gf\_log}[b]) \bmod 255], \quad a, b \neq 0 \tag{15}$$

Inverse. The multiplicative inverse of $a \in GF(2^8) \setminus \{0\}$:

$$a^{-1} = \text{gf\_exp}[255 - \text{gf\_log}[a]] \tag{16}$$

8.2 Capacity Formula

The total data capacity C_{bits} of an Ocode with outer radius R and inner data radius r_0 is:

$$C_{\text{bits}} = \sum_{k=0}^{\lfloor (R - r_0)/\Delta r \rfloor} \left\lfloor \frac{2\pi(r_0 + k\Delta r)}{\delta} \right\rfloor \tag{17}$$

For the default parameters ($r_0 = 85$, $\Delta r = 10$, $\delta \approx 5\,\text{px}$), a code with $R = 200$ holds approximately $C_{\text{bits}}/8 \approx 200$ data bytes before ECC overhead.

8.3 Rotational Symmetry and Sync

Unlike QR codes which rely on three fixed-position finder patterns, Ocode uses a single sync gap at a fixed absolute angle ($\theta = 0$). The decoder need only scan the perimeter of the outer ring to locate this gap; because the container ring is a complete circle, this scan is $O(n)$ in the perimeter pixel count and is computationally trivial.

9 Comparison with QR Code

Property	**QR Code**	**Ocode**
Shape	Square grid	Concentric arcs
Coordinate system	Cartesian (i, j)	Polar (r, θ)
Finder pattern	3 square alignment squares	1 bullseye + 1 sync gap
Error correction	Reed-Solomon (ISO 18004)	Reed-Solomon (custom GF)
ECC levels	L, M, Q, H	L, M, Q, H
Rotation detection	Orientation patterns	Sync gap perimeter scan
Output format	Raster / SVG grid	SVG arc paths
Dependencies	ISO/IEC 18004 spec	Zero (self-contained)
Visual aesthetic	Pixel grid	Flowing circular arcs

10 Conclusion

Ocode demonstrates that the well-understood mathematical machinery behind QR codes — Reed-Solomon error correction over $GF(2^8)$, finder patterns, rotational synchronisation — can be re-expressed in a radially-symmetric coordinate system to produce a format that is simultaneously functionally equivalent and visually distinct.

The all-client-side architecture ensures that encoded data never leaves the user's device. The zero-dependency GF implementation means the library can be embedded anywhere TypeScript or JavaScript runs without pulling in external packages.

Future directions for the Ocode format include: structured data modes (URI, vCard) with a dedicated mode byte in the header; multi-layer Ocodes (two interleaved data spirals); and a native mobile scanner module leveraging camera APIs on iOS and Android.

Live demo: ys1374.github.io/Ocode
Source code: github.com/ys1374/Ocode
License: MIT